

Summary of things that have been done

Overview

Successfully built, compiled, and verified the foundational UVM testbench for the synchronous FIFO. The environment is now fully capable of driving constrained-random traffic, monitoring transactions, checking data integrity, and collecting functional coverage.

Key Accomplishments

1. Complete UVM Architecture Built

Developed and integrated all 15 necessary testbench files from scratch:

- **Interfaces & Top:** `fifo_if.sv` , `fifo_top.sv`
- **Agent Components:** `fifo_agent.sv` , `fifo_sequencer.sv` , `fifo_driver.sv` , `fifo_monitor.sv`
- **Data Objects:** `fifo_seq_item.sv` , `fifo_sequence.sv`
- **Verification IPs:** `fifo_scoreboard.sv` (with golden model), `fifo_coverage.sv` (with transition & illegal bins), `fifo_sva.sv` (SystemVerilog Assertions)
- **Environment & Test:** `fifo_env.sv` , `fifo_test.sv`
- **Build System:** `fifo_pkg.sv` , `Makefile` (targeting QuestaSim)

2. First Successful Simulation (Random Stress Test)

Executed `fifo_random_test` , injecting ~2000 randomized transactions into the DUT.

```
eurekmeister@eurekmeister-Thinkinator: /mnt/4cd7130a-027f-496a-87a6-6cc6aded2046/Q2/rl-uvn-hardware-trojan/rl-uvn-hw-trojan
# Loading /mnt/faffib38-15c0-4dbc-a705-c0f65b2438a1/toolz/Qsim/questasim/uvn-1.1d/linux_x86_64/uvn_dpi.so
# run -all
# -----
# UVM-1.1d
# (C) 2007-2013 Mentor Graphics Corporation
# (C) 2007-2013 Cadence Design Systems, Inc.
# (C) 2006-2013 Synopsys, Inc.
# (C) 2011-2013 Cypress Semiconductor Corp.
# -----
# ***** IMPORTANT RELEASE NOTES *****
#
# You are using a version of the UVM library that has been compiled
# with 'UVM_NO_DEPRECATED undefined.
# See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
# You are using a version of the UVM library that has been compiled
# with 'UVM_OBJECT_MUST_HAVE_CONSTRUCTOR undefined.
# See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
# (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(+struct)
# UVM_INFO @ 0: reporter [RNTST] Running test fifo_random_test...
# UVM_INFO ./uvn/fifo_uvm/fifo_driver.sv(45) @ 0: uvm_test_top.env.agent.driver [fifo_driver] Driving reset...
# UVM_INFO ./uvn/fifo_uvm/fifo_driver.sv(55) @ 55000: uvm_test_top.env.agent.driver [fifo_driver] Reset released.
# UVM_INFO verilog_src/uvn-1.1d/src/base/uvn_objection.svh(1267) @ 20045000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ./uvn/fifo_uvm/fifo_coverage.sv(52) @ 20045000: uvm_test_top.env.coverage [fifo_coverage] Coverage: 100.0%
# UVM_INFO ./uvn/fifo_uvm/fifo_scoreboard.sv(118) @ 20045000: uvm_test_top.env.scoreboard [fifo_scoreboard]
# =====
# SCOREBOARD SUMMARY
# Total transactions : 1999
# Data matches (1.1 FIFO_data_out check): 844
# Data mismatches: 0
# Flag mismatches (1.2): 0
# =====
```

Simulation Log:

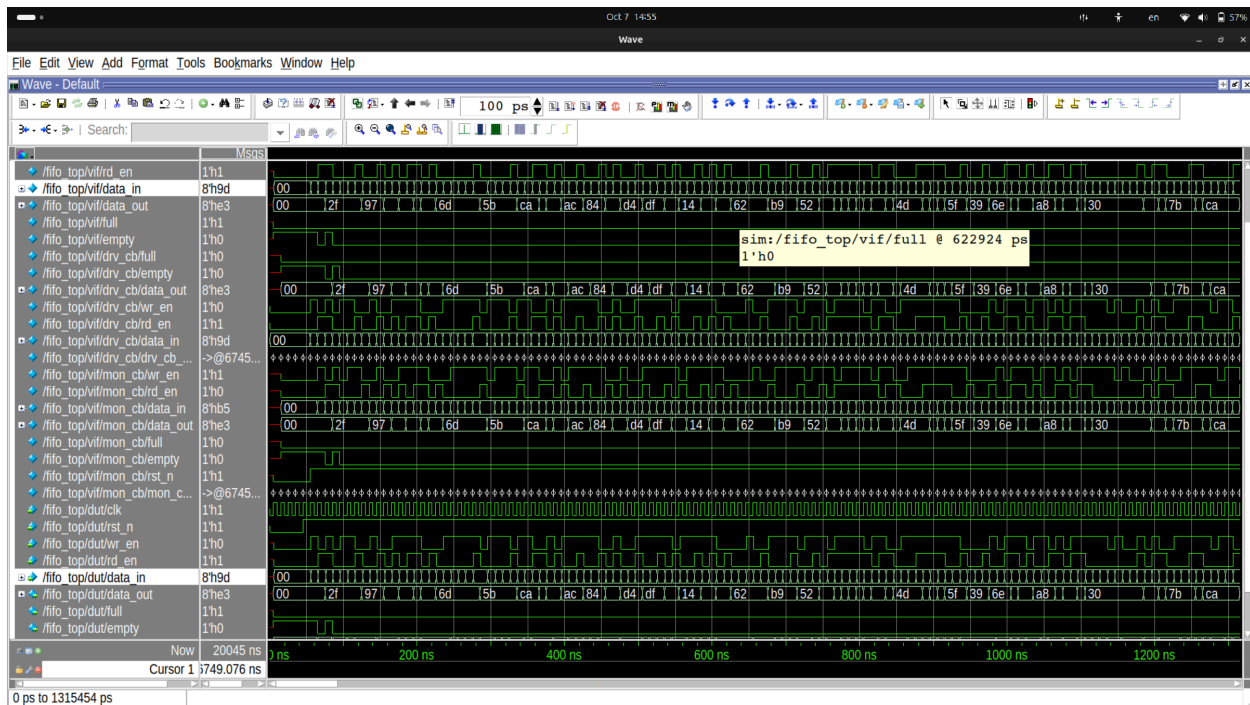
Results Breakdown:

- **Total Transactions:** 1999 executed perfectly.
- **Data Integrity:** 844 valid reads matched the golden model exactly (0 Mismatches).
- **Flag Integrity:** `full` and `empty` signals behaved exactly as predicted by the reference occupancy (0 Mismatches).
- **Coverage:** Reached **100.0%** functional coverage on all defined basic bins and transitions.

3. Waveform Visualization

Successfully dumped and visualized the VCD waveforms in the QuestaSim GUI.

Waveform Capture:



Observations:

- `wr_en` and `rd_en` toggle correctly according to the weighted random constraints (60/40 splits).
- The `full` and `empty` flags assert at the correct boundaries without causing sequence failures.
- Random 8-bit `data_in` values are successfully propagated to `data_out` after a 1-cycle read latency.

Next Steps: Directed Testing

With the random infrastructure proven, the next phase focuses on targeted verification scenarios:

- **T01:** Reset Sequence (mid-traffic reset recovery)
- **T02:** Single Write / Single Read behavior
- **T05:** Simultaneous Read & Write when partially full
- **T06:** Write when Full (Overflow rejection check)
- **T07:** Read when Empty (Underflow rejection check)